AstraZero: An AlphaZero Chess Engine Trained on $60 of Rented GPU Time

Technical report — 3 August 2026

Abstract

We reimplement AlphaZero for chess from scratch and train it to 39,404 self-play games across three cloud platforms on a total budget of roughly $60 in credits, plus a free Kaggle allocation. The engine improves measurably and monotonically: generation 21 defeats generation 14 by +255 Elo, generation 33 defeats 21 by +80 Elo, and generation 52 defeats 33 by +436 Elo (36–2–2 over 40 games, $p < 0.0001$).

The two largest effects we measured were not architectural. The first was a **labelling defect**: 32% of self-play games ended in draws while one side held a material advantage, because the 50-move and repetition rules fire long before a weak engine can convert. Those games taught the value head that material is worthless. Adjudicating them on material — the single piece of human chess knowledge in the system, applied only as a training label — produced the +255 Elo jump, the largest in the project.

The second was **infrastructure economics**. MCTS tree descent is single-threaded Python, so self-play throughput scales with CPU cores rather than GPU class; an RTX 4090 produced only 16% more games per hour than a T4. Running one self-play process per core cut cost per game by 5.8×. We further measured that per-container throughput scales as approximately N^0.35 in process count, which makes the intuitive optimisation — packing all cores onto one GPU container — 28% *worse* per game than fanning across several.

We report all measurements, including the ones that contradicted our predictions.

1. Introduction

AlphaZero learns to play chess with no human knowledge beyond the rules: a neural network proposes moves and evaluates positions, Monte Carlo tree search sharpens both, and the sharpened output becomes the next training target. DeepMind trained it on 44 million games using 5,000 TPUs.

This project asks a narrower question: **what does that algorithm actually produce at four orders of magnitude less compute, and where does the money go?**

The answer to the second half turned out to be more interesting than the first. Most of our gains came from finding defects in the training signal and in how compute was purchased — not from anything about the network.

1.1 Constraints

- No local GPU. All training on rented or free cloud compute.
- Sessions intermittent — hours at a time, days apart, with no loss of progress.
- Total spend roughly $60 across Beam and Modal, plus Kaggle's free 30 GPU-hours/week.

The intermittency drove the most consequential design decision, described in §2.4.

2. System

2.1 Network

A residual convolutional tower with policy and value heads, **12.15M parameters**:

```
input     8 × 8 × 119   (8-ply history, piece planes, castling, repetition, colour)
tower     8 residual blocks × 128 filters
policy    32 channels → 4,672 logits
value     8 channels → 128 hidden → scalar in [-1, 1]
```

The 4,672-move encoding is AlphaZero's: 8×8 origin squares × 73 planes (56 queen-like moves, 8 knight moves, 9 underpromotions). Illegal moves are masked before the softmax rather than learned against.

2.2 Search

PUCT MCTS with no rollouts — the value head replaces them entirely:

```
score(s,a) = Q(s,a) + c_puct · P(s,a) · √ΣN(s,·) / (1 + N(s,a))
```

Searches for N independent games run in lockstep, one leaf per game per network call, so the GPU sees a batch of N positions instead of one. Unvisited children take `parent_value - fpu_reduction` (first-play urgency) rather than zero, which otherwise makes every unexplored move look drawn.

Two bugs here cost real time and are worth recording. Checking the stop flag at simulation 0 left the root unexpanded and returned a move with no visits at all; the guard now runs only at `simulation > 0 and simulation % 8 == 0`. And the value backed up from a terminal node is in the *mover's* frame, so it must not be negated — the opposite of the child-value case three lines above it.

2.3 Training targets

Each position yields two targets:

- **policy** — the MCTS visit distribution, stored CSR-sparse. Dense storage would be 4.7 GB per training window at chess scale.
- **value** — the game's eventual result from that player's perspective.

2.4 The replay buffer stores games, not tensors

Shards hold move lists and visit counts, not encoded positions. This was the single most useful structural decision:

- **~0.07 MB per shard of 16 games**, so the entire history moves between platforms in seconds. We migrated the run Beam → Modal → Kaggle → Beam without loss.
- **Retroactive relabelling is possible.** The adjudication fix in §4 was applied to 7,115 already-played games by re-deriving their value targets. With encoded tensors those games would have been unusable and the fix would have cost a full retrain.
- **Encoding changes don't invalidate history.**

Every write is temp-file-then-rename, so a container killed mid-write cannot corrupt the checkpoint it would resume from.

2.5 Resumability

Checkpoints carry model weights, **optimizer state**, LR schedule position, and RNG state. Resuming from a stripped checkpoint restarts Adam's moment estimates from zero; after 25,200 steps that is a real regression, and every cloud export path we built stripped optimizer state by default for size. This is a trap worth flagging explicitly in any similar system.

---

3. Infrastructure

The run trained on three platforms with the same code:

Platform	Role	Cost
Beam	fan-out self-play, RTX 4090	~$30
Modal	fan-out self-play, mid-project	~$30
Kaggle	free T4, 30 GPU-h/week	free

Portability came from the compact buffer format (§2.4) and from avoiding platform primitives. In particular, **self-play workers are spawned as subprocesses, not via** `multiprocessing`. Serverless runtimes hold open gRPC file descriptors, and `spawn` tries to pass the parent's descriptor table to each child — on Beam this fails outright with `bad value(s) in fds_to_keep`. A fresh interpreter inherits nothing and works identically everywhere.

Each process writes its own shard. There is no coordination and no locking, which is what lets many containers share one volume safely.

---

4. Finding 1 — the value head was materially blind

4.1 Symptom

By generation 14 the engine had a low value loss (0.194) and played like a beginner. Querying the value head directly on constructed positions revealed why:

```
                    gen 21 (before)
      down a queen        -0.188
      down a rook         -0.026
      level               +0.192
      up a queen          +0.212
```

The spread across ±a queen was **0.426**, and earlier it had been **0.072** — the value head was emitting a near-constant regardless of material.

4.2 Cause

32% of self-play games ended drawn with one side a piece or more ahead. A weak engine shuffles; the 50-move rule and threefold repetition fire; the game is scored 0. The network correctly learned that being a queen up predicts a draw — because in its own games, it did.

Low value loss was *caused* by the defect. Predicting ≈0 everywhere scores well when most labels are 0.

4.3 Fix

Games ending by 50-move rule, repetition, or move cap are scored on material if the imbalance is ≥ 5 points. Stalemate and insufficient material are untouched — those are genuinely drawn.

This is the only human chess knowledge in the system, and it enters solely as a training label. Search and network are unchanged.

Applied retroactively to 7,115 games.

4.4 Result

```
                     loss    policy   value
      gen 15        2.699    2.528    0.171    ← best loss ever recorded
      gen 16        2.879    2.507    0.372    ← adjudication enabled
```

Loss got worse and the engine got much stronger (+255 Elo). The old targets were easy and uninformative; the new ones are hard and carry signal. Value spread rose 0.072 → 0.426, and self-play draws fell from 32% to 12.5%.

This is the clearest result in the project: **loss is a proxy for fitting the search, not for playing well.**

---

5. Finding 2 — self-play is CPU-bound

MCTS tree descent is pure Python (`python-chess` move generation dominates). One self-play process saturates exactly one core no matter how large the GPU.

Measured evidence:

- Throughput was nearly independent of search batch width.
- An **RTX 4090 produced only ~16% more games per hour than a T4**.

A 4-core container running one self-play process wastes three cores *and* most of the GPU while paying for all of it. Running one process per core reduced cost per game by **5.8×** — the largest single cost lever in the project.

---

6. Finding 3 — container shape, and a prediction that failed

Beam prices a container as a fixed GPU rate plus per-core and per-GiB rates:

```
RTX 4090   $0.690/hr    (the cheapest GPU offered)
core       $0.045/hr
RAM        $0.008/hr per GiB
```

Since cores produce games and the GPU is barely used, the apparent optimisation is to pack every core onto **one** container and rent one GPU instead of several. We predicted 1,044 games/$ against 569 for the fanned-out shape — an 83% improvement.

Measured:

Shape	Games/hr	$/hr	$/1,000 games
1 × 8	631	$1.11	**$1.76**
3 × 8	1,893	$3.33	**$1.76**
1 × 30	1,003	$2.27	$2.26
3 × 8 (final)	2,073	$3.33	**$1.61**

Per-container throughput scales as approximately **N^0.35** in process count: 8 processes yield 79 games/hr each, 30 yield 33 each. The GPU rent saved was smaller than the throughput lost, making the "optimised" shape **28% worse per game**.

The correct rule is the opposite of the intuition: **scale by adding containers, not cores**, with roughly 6–8 processes per container.

This cost about $1.20 and 45 minutes to discover, because we changed a working configuration on the strength of a cost model rather than probing first. A 12-minute probe would have shown it.

---

7. Evaluation

7.1 Why not loss

Section 4 establishes that loss can move opposite to strength. We use three signals instead, in descending order of trust:

- **Gate matches** — head-to-head at equal search depth. Ground truth.
- **Value spread** — the value head's range across constructed material imbalances, with no search. Has predicted the gate-match direction twice.
- **Self-play draw rate** — falling means more decisive play.

7.2 Policy loss has a measured floor

Cross-entropy against the MCTS visit distribution cannot fall below that distribution's own entropy. Measured over 50,199 plies of stored 200-simulation targets:

```
        mean policy-target entropy = 1.736 nats   (median 1.725)
```

Generation 52's policy loss of 2.313 therefore sits **0.58 nats above an achievable floor**, not 2.313 above zero. The floor is also configuration-dependent: raising search from 100 to 200 simulations dropped policy loss from 2.480 to 2.421 immediately, because deeper search produces a more decisive — lower-entropy — target.

Value loss has a floor too. Predicting 0 everywhere on generation 20's games scores 0.864, so generation 52's 0.231 explains roughly **73% of outcome variance**. Zero is unreachable: from the opening the result is genuinely uncertain.

7.3 Results

Gate matches (40 games, 200 simulations, 12 random opening plies):

Match	W–L–D	Score	Elo	95% CI
gen 21 vs 14	30–5–5	0.813	**+255**	—
gen 33 vs 21	20–11–9	0.613	**+80**	[–13, +186]
gen 52 vs 33	36–2–2	0.925	**+436**	[+303, +1184]

The +80 result is real but not tightly resolved at 40 games. The +436 result is unambiguous: $z = 11.3$, $p < 0.0001$, with draws collapsing to 5%.

One caveat on the final match. Generation 33 was trained on 100-simulation targets; generation 52 on 200-simulation targets; both were evaluated at 200. The deeper search at test time therefore matches generation 52's training distribution more closely. This is a real effect and not separable from strength here — but it means the +436 figure reflects both better play and better adaptation to the evaluation condition.

Value head diagnostics (no search):

	gen 21	gen 33
Correlation with material	+0.910	+0.925
Spread across ±queen	0.426	**0.638**
Tactics solved	4/5	4/5

The improvement is asymmetric and worth noting: generation 33 rates being down a queen at –0.748 versus generation 21's –0.188, but both are flat on the upside — up a knight, up a rook and up a queen land within 0.01 of each other. **The engine has learned to fear material loss much better than it has learned to exploit material gain.** That follows from adjudication: being down 5+ points reliably scores as a loss, while converting an advantage still requires technique the engine lacks.

Final training session (Beam, 18 generations, 6,896 games, ~4 hours, ~$16):

```
            loss    policy   value   holdout v-gap
gen 35   2.756    2.404    0.353      +0.082
gen 44   2.650    2.373    0.277      +0.095
gen 52   2.544    2.313    0.231      +0.062
```

Value loss fell 35% while the **holdout gap narrowed**. That combination matters: falling loss with a widening gap would indicate memorisation of the training window. Falling loss with a narrowing gap is generalisation.

---

8. Limitations

- **39,404 games is roughly 0.09% of AlphaZero's 44 million.** The engine plays at perhaps club-beginner level and hangs pieces in sharp positions.
- **Neither generation 21 nor 33 finds a back-rank mate in 1** at 400 simulations. Positional instinct about material is developing faster than concrete calculation.
- **Value calibration is poor outside the training distribution.** Both checkpoints misjudge bare-king endgames — generation 21 by +0.19, generation 33 by –0.22 — because adjudication ends such games long before they arise. Ordering and spread are meaningful; absolute values are not.
- **40-game gate matches are statistically thin**, roughly ±110 Elo at 95%.
- **No comparison against an external engine.** All Elo figures are internal and say nothing about absolute strength.
- **Adjudication at 5 points is unvalidated.** It is plainly better than 0; we never tested 3 or 7.

---

9. What we would tell someone starting this

- **Look at what your training labels actually say.** The largest gain here came from noticing that a third of games taught the network something false. No architectural change came close.
- **Loss is not strength.** The single best result arrived alongside a loss increase.
- **Measure before optimising, even when the arithmetic is convincing.** Two cost models in this project were confidently wrong in the same direction.
- **Store games, not tensors.** It made a retroactive fix to 7,115 games possible and turned platform migration into a file copy.
- **Silent success is worse than a crash.** Several failures here — a corrupted upload, a session training on an empty buffer, a duplicate run — looked healthy in the logs. Every guard that now refuses to proceed quietly was added after one of them.
- **Print the actual state.** Two rounds of reasoning about where a dataset was mounted were resolved in one run by listing the filesystem.

---

10. Reproducibility

```
git clone https://github.com/Madushan996/AstraZero.git
cd AstraZero && pip install -r requirements.txt

python train.py train --run runs/chess --game chess --minutes 60
python train.py eval  --run runs/chess --candidate 10 --baseline 0 --games 40
python tactics.py --run runs/chess
```

129 tests pass, including a Connect 4 end-to-end run that validates the full loop in minutes rather than hours. Chess-scale correctness is checked separately — encoding round-trips, adjudication boundaries, and MCTS behaviour on forced positions.

Configuration for the results above:

```
net       8 blocks × 128 filters, 12.15M parameters
search    200 simulations, c_puct 1.5, Dirichlet α 0.3 on the root
training  400 steps/generation, batch 512, Adam lr 2e-3
buffer    40,000-game window, ~3,200 games sampled per step
adjudicate_material_at = 5
```